



Other user-defined data type

Union



Syntax of union

- Follow same syntax as structures but members share storage
 - Allocate memory large enough to hold the largest member
- **union** keyword is used to create a union

```
union union_name {  
    data_type member1;  
    data_type member2;  
    // ... more members  
};
```

int i: 4 bytes; float f: 4 bytes; char c[20]: 20 bytes
The compiler looks for the largest member, which is c[20] at 20 bytes.
It will allocate one 20-byte block of memory for the entire union.
If you use i, it will use the first 4 bytes of that 20-byte block.
If you use f, it will also use the first 4 bytes of that 20-byte block.
If you use c, it will use the entire 20-byte block.

Create union

- Following examples shows how to create a union

```
union MyUnion {  
    char c;  
    int i;  
    float f;  
    double d;  
};
```

Store values to union members

```
union MyUnion {  
    char c;  
    int i;  
    float f;  
    double d;  
};
```

- You can store value to only one member of the union

```
union MyUnion u;
```

```
u.i = 7;
```

Access and print values of union members

- You can store value to only one member of the union

`u.i = 7;`

- Print values of the union members

```
printf("%c\n", u.c);  
printf("%d\n", u.i);  
printf("%f\n", u.f);  
printf("%lf\n", u.d);
```

```
.  
7  
0.000000  
354657452
```

When do we use union?

- To use memory efficiently to represent different data types
- Programmer's responsibility to choose the right one
- Use case
 - low-level programming

Ex: Union



- A message system stores one of the three types of message: text, image and audio.
 - For text messages , it store the entire text
 - For image and audio messages, it stores the file name

Take user input and accordingly create the corresponding message

Ex: Steps to solve the messaging programming task



Steps

- 1: Code the TEXT, IMAGE and AUDIO message as 0, 1, and 2 respectively.
 - Create enumerator for the three types of messages – TEXT, IMAGE and AUDIO
2. Create union to store the message content.
3. Create a structure message whose
 - first member is the message **type** and
 - second member is the message **content**

Its Purpose: This is the raw data container. Since a message can only be one thing at a time (either text or an image or audio), it's a waste of memory to store all three. The union lets you store just one of them in a single, memory-efficient slot.

```
#include<stdio.h>
#include<string.h>
// Define the message types
enum MessageType {
    TEXT,
    IMAGE,
    AUDIO };
// Union to store different types of message content
union MessageContent {
    char text[256];
    char imageFileName[128];
    char audioFileName[128];
};
// Structure to represent a message
struct Message {
    enum MessageType type;
    union MessageContent content;
};
```



Ex: Steps to solve the messaging programming task

Steps

- 1: Code the TEXT, IMAGE and AUDIO message as 0, 1, and 2 respectively.
 - Create enumerator for the three types of messages – TEXT, IMAGE and AUDIO
2. Create union to store the message content.
3. Create a structure message whose
 - first member is the message **type** and
 - second member is the message **content**
4. Take user input and create the corresponding message

```
int main() {
    int msgType;
    struct Message message;
    printf("Enter the message type (0 - 2)\n");
    scanf("%d", &msgType);
    switch(msgType) {
case TEXT: // Create text message
message.type = TEXT;
strcpy(message.content.text, "Hello World");
break;
case IMAGE: // Create Image message
message.type = IMAGE;
strcpy(message.content.imageFileName,
"image.jpg");
break;
case AUDIO: // Create audio message
message.type = AUDIO;
strcpy(message.content.audioFileName,
"audio.mp4"); break; default:
printf("Erroneous input");
return 0;
}
```

```
switch(msgType) {
case TEXT:
    printf("Text Message: %s\n",
message.content.text);
    break;
case IMAGE:
    printf("Image Message: %s\n",
message.content.imageFileName);
    break;
case AUDIO:
    printf("Audio Message: %s\n",
message.content.audioFileName);
    }
return 0;
}
```

Files

Open and close files

- . *Write (or Overwrite): This is like erasing the notebook (or starting at page 1) and writing new content. Any old content is destroyed.*
- . *Read: This is opening the notebook to page 1 and reading what's already there. You don't change anything.*
- . *Append: This is finding the last written sentence in the notebook and adding your new notes after it. The old content is preserved, and new content is added to the end.*

Definitions

- File: It can be abstractly thought of as a stream of bytes. It is used for persistent storage.
- Files have a name. They can be written to, or read from, or appended to.
- In order to access the file, it must be opened. After finishing use of the file we close it.

C's view of a file

- File: Sequence of bytes
 - Ends with end of file (EOF) marker
 - EOF constant defined in stdio.h

0	1	...	n	
data	data	..	data	EOF

“r” (read): Opens an existing file for reading. If the file doesn't exist, it fails (returns `NULL`).

“w” (write): Creates a new file for writing. If the file already exists, it erases all its content.

“a” (append): Creates a new file for writing or opens an existing one. All new data is written to the end of the file, preserving existing content.

“r+”: Opens an existing file for both reading and writing.

“w+”: Creates a new file for reading and writing (or erases an existing one).

File handling operations

- Create a new file
- Open an existing file `Open()`
- Write to a file
- Read from a file
- Move to a specific location in a file
- Closing a file `Close()`

fscanf(FILE stream, ...): Reads formatted text, just like scanf.*
fgets(char str, int n, FILE* stream): Reads a full line of text (up to n-1 characters).*

fgetc(FILE stream): Reads a single character.*
fread(void ptr, size_t size, size_t count, FILE* stream): Reads raw binary data.*

```
// To OPEN an existing file for reading:  
FILE* f_read = fopen("my_data.txt", "r");
```

```
// To CREATE a new file for writing:  
FILE* f_write = fopen("new_report.txt", "w");
```

```
// Always check if the open operation failed!  
if (f_read == NULL) {  
    printf("Error: Could not open my_data.txt!\n");  
}
```

File pointer

- **FILE** is a structure
 - contains information about the file
- File pointer declaration
`FILE *ifp;`
 - Creates a file pointer called ifp.
 - FILE data type defined in `stdio.h`

Write to a File

`fprintf(FILE* stream, ...)`: Writes formatted text, just like `printf` but to a file.

`fputs(const char* str, FILE* stream)`: Writes a string.

`fputc(int char, FILE* stream)`: Writes a single character.

`fwrite(void* ptr, size_t size, size_t count, FILE* stream)`: Writes raw binary data (like structs or int arrays).

File pointer

- File descriptor
 - Location in memory associated with the input or output file stream
 - A file descriptor is an integer that represents an open file within a process.
- Declaring the file pointer variable does not associate a file descriptor with them.
 - A file pointer is a variable that stores address of a structure containing information about the file, such as its current position, buffer status, etc.

Creating a file

- Syntax to open a file
`FILE *fopen(const char *file_name, const char mode);`
- Accepts two parameters
 - file_name: String type, the name of the file to be created/opened
 - mode: Mode of file access
- Returns NULL if it fails to open a file

Ex: Creating a File

- `FILE *ifp;`
- `ifp = fopen("Test.txt", "w");`
 - Opens a file named Test.txt for writing
 - Returns NULL if open fails.

fopen modes



Mode	Description
r	open existing file for reading
w	Create file for writing. If file exists contents discarded
a	append; open an existing file for reading/writing
r+	Open an existing file for update (read/write)
w+	Create a file for update
a+	Open/create a file for update
rb, wb,ab	Read, write, append to binary files
rb+, wb+,ab+	Open, create, append binary file for update



fopen()



- Opening a file for reading that cannot be read or does not exist will fail and a NULL pointer will be returned.
- Opening a file for writing creates a new file if does not exist and overwrites if exists.

Testing for successful open

- `fopen()` does not guarantee that the file will be opened successfully

```
FILE *ifp;  
ifp = fopen("Test.txt", "r");  
if (ifp == NULL) {  
    printf("Couldn't open file!!");  
    exit(1);  
}
```

Closing a file

- `int fclose(FILE *fp)`
 - If succesful returns zero otherwise EOF
 - Empty buffers and breaks all connection to the file

Ex: Closing a file

```
FILE *ifp = fopen("Test.txt", "r");  
// Read and write to file  
fclose(ifp);
```

- Closes the file pointed to by ifp and releases the file descriptor space and I/O buffer memory.



Files

Writing to files



Functions to write to a file

- `int fprintf(FILE *stream, const char *format, ...)`
 - Write formatted data to a file
- `int fputs(const char *str, FILE *stream)`
 - Write strings to a file
- `size_t fwrite(const void *ptr, size_t size, size_t count, FILE *fp)`
 - Write block of data to a file
- `int fputc(int character, FILE *file_pointer);`
 - Write characters to a file

Writing to Files -fprintf

- `int fputs(const char *str, FILE *stream)`
 - Writes a string to the specified stream up to but not including the null character.
- Example

```
FILE *fp;  
fp = fopen("file.txt", "w+");  
fprintf(fp, "%s", "This is MA511.");
```

Writing to Files -fputs

This is the key line. fputs stands for "File Put String."



- `int fputs(const char *str, FILE *stream)`
 - Writes a string to the specified stream up to but not including the null character.
- Example

```
FILE *fp;
fp = fopen("file.txt", "w+");
fputs("This is MA511.", fp);
```

Unlike puts (which prints to the console), fputs does not automatically add a newline character (\n) at the end. The file will contain the literal text 'This is MA511' . and nothing more.

```
fprintf(fp, "%s", "This is MA511.");
```

. This is the "File Print Formatted" function. It's powerful and can handle complex formatting with %d, %f, %s, etc.

```
fputs("This is MA511.", fp);
```

. This is the "File Put String" function. It's simpler and more direct. It's often slightly more efficient if all you are doing is writing a plain string.

Writing to files: fwrite

- `size_t fwrite(const void *ptr, size_t size, size_t count, FILE *fp)`
 - `ptr` – Pointer to the array of elements to be written
 - `size` – size of array element to be written in bytes
 - `count` – number of elements to be written
 - `fp` – file pointer

ptr: "What data do I want to write?" (e.g., the name of your array)

size: "How big is one item in my array?" (e.g., `sizeof(int)` or `sizeof(MyStruct)`)

count: "How many of these items am I writing?" (e.g., the length of the array)

fp: "Which file am I writing to?"

Ex: Writing to files: fwrite

```
#include<stdio.h>

int main () {
    FILE *fp = fopen( "file.txt" , "w" );
    char str[] = "DA 104:C programming";

    fwrite(str, sizeof(str), 1, fp );

    fputs("\n", fp);

    fclose(fp);
    return(0);
}
```

You will have a file named users.dat. If you try to open this file in a simple text editor (like Notepad), it will look like garbage (e.g., 'Alice Bob Charlie).

This is because it is not a text file. It's a binary file containing the raw memory representation of your structs. The only way to read this file correctly is to use its sister function, fread(), in another C program that also knows the definition of the 'User struct.

Ex: Writing to files: fwrite

```
#include<stdio.h>

int main () {
    FILE *fp = fopen( "file.txt" , "w" );
    char str[] = {'D', 'A', '1', '0', '4' };
    fwrite(str , sizeof(char) , 2, fp );

    fputs("\n", fp);
    fclose(fp);
    return(0);
}
```

Output: DA

Writing to Files -fputc

- `int fputc(int char, FILE *stream)`
 - Writes single character at a time to file
 - Advances the file pointer

- Example

```
FILE *fp;  
fp = fopen("file.txt", "w+");  
char string[]="Testing fputc";  
for (int i=0;string[i]!='\0';i++)
```

```
    fputc(string[i],fp);
```

```
fclose(fp);
```

Output : Testing fputc

fgetc vs. getc: They are nearly identical. fgetc is guaranteed to be a function. getc can be implemented as a macro, which might be slightly faster but can have side effects if used with complex arguments (e.t., `getc(fp++)`, which you shouldn't do).

Rule of thumb: When in doubt, use fgetc.

Files

Reading from files

Functions to read from a file

- `int fscanf(FILE *stream, const char *format,...)`
 - Reads formatted data from file
- `int fgets(char *str, int n, FILE *stream)`
 - Reads a line from the file and stores it in the string pointed to by str.
- `size_t fread(void *ptr, size_t size, size_t count, FILE *fp);`
 - Reads a block of data from the file
- `int fgetc(FILE *stream)`
 - Reads a character from the file
- `int getc(FILE *file_pointer);`
 - Similar to `fgetc`

Read one character

Read one character

fgets : A line of text

stops-> A newline(\n) is read, n-1 chars are read, or EOF is hit

Use-> Reading a file line-by-line(most common, safest)

fread : A block of binary data

*stops-> The requested no. of bytes(size * count) is read, or EOF*

Use-> Reading binary files(.dat, images, compiled structs)

Reading from files - fscanf

- `int fscanf(FILE *stream, const char *format,...)`
- Example
`FILE *ifp;`
`ifp = fopen("Test.txt","r");`
`fscanf(ifp,"%d %d", &a,&b);`

"%d %d": This is the format string. It's a "template" that tells fscanf what to look for.

%d: Look for an integer.

(space): Look for and skip over any amount of whitespace (spaces, tabs, or newlines).

%d: Look for another integer.



Reading from files - fgets

- `int fgets(char *str, int n, FILE *stream)`
 - Used for reading a string value with spaces
 - Reads a string of size `n` into the string pointed to by `*str`

- Example

```
#define SIZE 120
```

```
FILE *ifp;
```

```
char line[SIZE];
```

```
ifp = fopen("Test.txt","r");
```

```
fgets(line,SIZE,ifp)
```

line: The destination buffer to store the text.

SIZE (120): The maximum size of the buffer. This is the critical safety feature of fgets. It will read at most SIZE - 1 (or 119) characters, always leaving room for the null terminator.

ifp: The file to read from.

Reading from files - fread

- `size_t fread(void *ptr, size_t size, size_t count, FILE *stream)`
 - `ptr` – Pointer to a block of memory with a minimum size of `count*size` bytes. *The Destination*
 - `size` – This is the size in bytes of each element to be read.
 - `count` – Number of elements each of size “size”
 - `fp` – file pointer

*Total bytes read = size * count*

fread: This is like photocopying a page. It doesn't understand the content at all. It just makes an exact byte-for-byte copy of a block of data from the file directly into a block of memory.

You must use fread for any non-text file, such as images, audio files, or, most commonly, data 'structs' that you saved earlier using 'fwrite'.

Ex: Write and read

```
#include<stdio.h>
```

```
int main () {
```

```
    FILE *fp = fopen( "file.txt" , "w+" );
```

```
    char str[] = "DA 104:C programming";
```

```
    char buffer[30];
```

```
    fwrite(str , sizeof(str) , 1, fp );
```

```
    fputs("\n", fp);
```

```
    fseek(fp, 0, SEEK_SET);
```

```
    fread(buffer, sizeof(str), 1, fp);
```

```
    printf("%s", buffer);
```

```
    fclose(fp);
```

```
    return(0);
```

```
}
```

```
#include <stdio.h>
```

```
int main() {  
    FILE *fp = fopen("file.txt", "w+");  
    if (fp == NULL) {  
        printf("Error opening file!\n");  
        return 1;  
    }  
}
```

```
char str[] = "DA 104:C programming";  
char buffer[30];
```

```
fwrite(str, sizeof(str), 1, fp);
```

```
fputs("\n", fp);
```

```
fseek(fp, 0, SEEK_SET);
```

```
fread(buffer, sizeof(str), 1, fp);
```

```
printf("%s", buffer);
```

```
printf("\n");
```

```
fclose(fp);
```

```
printf("Successfully read/n");
```

```
return 0;
```

```
}
```



Reading from files - fgetc

- `int fgetc(FILE *stream)`
 - Used to read single character at a time from file
 - After reading character, file pointer advances to next
- Example

```
ifp = fopen("Test.txt","r");
do {
char c=fgetc(ifp);
      if (feof(fp))
          break;
      printf("%c",c);
} while (1);
```

Test: EOF

- end-of-file indicator informs there are no more data to be processed.
- Number of ways to test for end-of-file condition. One is to use the `feof`

Test: EOF

- The other way to test for end-of-file condition is to use EOF

```
FILE *ifp;  
ifp = fopen("Test.txt", "r");  
while(fscanf(ifp, "%d", &a) != EOF) {  
    // do something  
}
```